

he following table outlines the options available to issue temporary trial licenses based on specific use cases:

Use Case	Pathway
-----	-----
Customer renewal is taking longer than expected	The sales AE (Account Executive) generates a one-off 21 day temporary renewal extension via SFDC
Customer renewal exceeds the additional 21 days	The sales AE can open a new Internal Request (IR) ticket with L&R support and request a trial subscription extension for up to 1 month
Customer renewal exceeds the additional 21 days + 1 month	The sales AE can open a new Internal Request (IR) ticket with L&R support who request approval via the ticket from the senior director of revenue @andrewmurray
New customer potential sale	The sales AE can request up to 1 month trial extension via an IR with L&R support.
New customer sale taking longer than 1 month	The sales AE generates a \$0 dollar opportunity in SFDC, then opens a new IR ticket with L&R support who request approval via the ticket from the senior director of revenue @andrewmurray

How to create an NFR (Not for resale) SaaS License

As of Feb. 19, 2025, partner NFR subscriptions) will be provisioned as standard GitLab.com subscriptions instead of as trial subscriptions. Ecosystem Operations manage this process and no longer requires assistance from the support team to provision. Reach out to global-ecosystem-programs-ops via slack with any questions.

SECTION: support/license-and-renewals/workflows/self-managed/_index.md

SECTION: support/license-and-renewals/workflows/self-managed/cloud-licensing.md

Cloud Licensing Overview

Cloud Licensing allows GitLab customers to activate paid subscription features on self-managed instances using an activation code, rather than managing license files and manually uploading them into their instance. By activating through Cloud Licensing, a customer's GitLab instance will periodically synchronize Subscription Data with GitLab.

In order to further encourage Cloud Licensing, the Strict Cloud Licensing project will include several iterations to encourage customers to use Cloud License activation codes rather than a Legacy License file to activate their self-managed instances. The following GitLab internal licensing terms page provides a technical definition of the current three types of licenses (Cloud Licensing, Offline and Legacy) associated with Cloud Licensing.

Strict Cloud Licensing

Effective July 7, 2022: Cloud Licensing will be enabled by default for all new and renewal customers, as detailed in the Strict Cloud Licensing roll out plan (except OSS, EDU or Start-Up products). Customers will no longer have the ability to download a license file from the Customers Portal. In addition, Offline licenses will now be available to allow customers with air-gapped or offline instances to receive the benefits of Cloud Licensing. In order to receive a Legacy License or Offline License, customers will need to follow the below outlined process.

Strict Cloud Licensing Process

NOTE: This only applies to standard and resold customers.

mermaid

graph TD;

A -->|IR| B;

B -->|No| C;

B -->|Yes| E;

C --> D;

D --> E;

A -->|Customer Ticket| F;

F -->|Yes| G;

F -->|No| H

H -->|Yes| G;

H -->|No| I;

A{Is the ticket an internal
request or a customer ticket?}

B{Has approval been granted
for the exception?}

C(Support uses
Support::L&R::Strict Cloud Licensing Sales Exemption Guidance
macro)

D(Sales has gotten exemption
approval)

E(Support generates legacy
license and replies on ticket
letting requester know)

F{Did the user purchase/renew
before July 2022?}

G(Support uses
Support::L&R::Strict Cloud Licensing - Legacy license generated
macro)

H{"Did they always purchase web direct?
· See FAQ 4"}

I(Support uses
Support::L&R::Strict Cloud Licensing - contact sales
macro)

Cloud Licensing Exemptions

Pre-Sale Exemption (Sales)

Customers who cannot activate with Cloud Licensing will need to obtain either an Offline Cloud License or a Legacy License. If this process is handled correctly at time of sale, no Support involvement is required.

By default, the TurnOnCloudLicensingc flag in SFDC will be set to Yes on every quote during the sales cycle. If the Sales Rep wants to opt the customer out, they should update the quote field Cloud Lic] Add Cloud Licensing Opt Out to either Legacy License File or Offline License. This in turn updates the value of TurnOnCloudLicensing to No or Offline respectively. Once the quote is submitted, Sales VP approval is required via the standard SFDC approval process. Once approved, the quote is synced to Zuora, which in turn creates and activates the subscription. In this scenario, the customer will receive their activation email with their license file attached and will also have the ability to download the license file from the [GitLab Customers

portal.

Post-Sale Exemptions (Support)

The Pre-Sale Exemption process is intended to be the only means of opt out, and should be used for all customers. However, there may be times where the account manager missed the opt-out during the sale, and the customer has now incorrectly received a Cloud License activation code they cannot use.

If a customer requires a Legacy License or an Offline License post-sale, the sales account manager must obtain Sales VP approval for a Cloud Licensing exemption. When approval is obtained and documented in SFDC for each opportunity, the account manager must then open a Support Internal Request for each opportunity. On the form, select either GitLab Support Internal Request for Global customers or GitLab Support Internal Request for US Federal customers and then Self-Managed License Related (for paid customers only) > Cloud Licensing exemption. Full instructions for the account manager can be found in Highspot (visible to Sales team only).

If the customer's subscription or trial/temp license has expired, and they require access during the approval process, the account manager must open an Internal Request with Support. Use either the GitLab Support Internal Requests for Global customers or GitLab Support Internal Requests for Federal customers request option and then select Extend an (almost) expired subscription in the Self Managed section. The support engineer should provide the customer with a trial license until post-sales opt-out approval is received and handled in the related Internal Request.

How to process a Cloud Licensing exemption internal request

1. Click on the Chatter with approval link then:

1. Verify the following:

- The chatter is on a Salesforce Opportunity page.
- The Opportunity is marked as Closed-Won (see FAQ 5 if it is not).
- The chatter thread includes an Approval message.
- The exemption approval is provided by a VP. Hover on the approver's name to see their title.

1. If any of the above requirements are not met, let the requester know what should be corrected and then close the Internal Request. Otherwise proceed to step 2.

1. While on the Opportunity page, find the related subscription details

1. Using the Sold To Email, search for the customer account in CustomersDot Customers page.

1. Click on the bookmark icon to navigate to the account's Zuora Subscriptions tab.

1. Confirm the subscription that should be updated by verifying the Subscription Name from step 2.

1. Go back to the IR ticket and note the GitLab Version value:

- If the version < 15.0, set the Cloud Licensing flag value to No.
- If the version >= 15.0, set the Cloud Licensing flag value to Offline.

1. Click Update.

1. Click the Impersonate tab.

1. Click Copy license key to clipboard. Using this method will ensure all license details are auto-populated correctly.

- If you see An error occurred... message, try to click the Copy license key to clipboard

once more.

- If it fails again, check the logs for the issue or ask for assistance.

1. Click the Back button on your browser to stop impersonating the customer account.

1. Navigate to the Licenses page.

1. Locate the new generated license:

- The license should be one of the first licenses at the top.

- If it is not, search for the license using the Sold To Email from step 2.

1. Resend the license

because the automatically generated license does not trigger license email notification.

1. Reply to the IR ticket with a link to the license and mark the ticket as Solved.

Note

The matrix below defines the impact of setting the Cloud Licensing flag value to Yes, Offline or No on the eligibility of three license types.

Cloud Licensing Flag value	Cloud License	Offline Cloud	Legacy License
-----	-----	-----	-----
Yes	Eligible	Not Eligible	Eligible
Offline	Eligible	Eligible	Not Eligible
No	Eligible	Eligible	Eligible

Find the related subscription details

There are a few ways to locate the subscription in a closed-won opportunity.

Refer to the alternative methods below for guidance on different cases.

Customer Subscription (CS-0000000000)

1. Scroll down on the Opportunity page to the Subscription Information section.

1. Click on the Customer Subscription value which has the format CS-0000000000 to go to Customer Subscription page.

1. Click the Current Zuora Subscription value to go to the Subscription page.

1. Note the Sold To Email and the Subscription Name from the Subscription page.

Quote

1. Scroll down on the Opportunity page to the Quotes section.

1. Locate the most relevant Quote which has the Status as Sent to Z-Billing.

- If all quotes have the Status as New, this indicates a web-direct purchase. For cases with multiple quotes that all have Status as New, use the quote that has been marked as Primary.

1. Open the quote.

- If you encounter a Content cannot be displayed... error, try using the Customer Subscription (CS-0000000000) process instead or ask for assistance.

1. Note the Sold To contact's email:

1. Scroll down to Sold to Contact in the quote details.

1. Hover on the name which opens a contact modal then right click on the email and copy the email address

1. Note the Subscription Name:

- Check the Subscription Name in the quote which is at the 4th row of the top section.
- If it is empty, or does not show up in CustomersDot, find the subscription by locating the customer account using the Sold To contact's email and verifying the subscription seat count matches the quote you opened.

Support FAQ

1. Which license type should I provide for an approved opt out?

Knowing whether to provide an Offline or Legacy License will depend on the customer's specific scenario and what they request, but the following may be helpful:

- Offline licenses are preferred by GitLab over Legacy Licenses as they allow the customer to more easily provide usage data. These work best for customers who cannot use a Cloud License because their instance is air-gapped or not connected to the internet. However, in order to use an Offline Cloud License, the customer must be on 15.0 or higher.
- Legacy licenses can be sent when a customer is unwilling or unable to upgrade to GitLab version 14.1 or higher, or is concerned about sharing Subscription Data.

2. Which workflow applies for reseller purchases?

For example: if a customer purchased after 2022-07-07 and needs a Legacy License, should we send them to their Account Manager to go through the exemption process, or do we treat them the same as web-direct and give them a Legacy License file, no questions asked?

Reseller customers should be directed to sales using the same exemption process as for sales-assisted purchases.

3. Can multiple license types be active for one subscription?

For example: a customer would like to use a Legacy License on a dev instance after activating their production instance with a Cloud License activation code.

We can generate a legacy license in this situation, but only after an exemption has been approved. Do not apply the exemption to the currently CL-enabled subscription. Instead, generate a legacy license duplicate of the existing license.

Please see this comment thread for greater context on this workaround.

4. How do I tell if a purchase was web direct?

Check the Created By value of either (1) the invoices for the specific purchase in the customer's Zuora account

(<<https://www.zuora.com/apps/CustomerAccount.do?method=view&id=<ACCOUNTID>>> or (2) the Subscription's Change History

(<<https://www.zuora.com/platform/subscriptions/><SUBSCRIPTIONID>>>):

- Web direct: If it is the Fulfillment API User (svczuorafulfillmentint@gitlab.com or rubenAPIproduction@gitlab.com).
- Not web direct: If it is either the Salesforce API User (svcZuoraSFDCintegration@gitlab.com) or

a GitLab team member's email (mostly it would be a Billing team member).

"New Business" web direct purchase opportunities in SFDC will often be created as ACCOUNT-NAME - Web Direct. The Initial Source field will also be filled as "Web Direct".

- Note: A sales-assisted renewal or add-on may still reflect Initial Source as "web direct", so you can further confirm:

Web direct purchases also create a Quote in the customer's Salesforce account. However, the Status of the Quote is almost always New.

- For Sales assisted and reseller purchases, you can confirm the details of the successful purchase by checking the Quote in the customer's Salesforce account with its Status set to Sent to Z-Billing.

5. What if the exemption is on an Opportunity that is NOT Closed-Won?

Cloud Licensing is required on a Closed-Won Opportunity because Support follows the Quote that has its Status as Sent to Z-Billing to confirm the license details that has been exempted.

- If the provided Opportunity is in an open stage, you can let the requester know they should use the pre-sale opt out process and then close the Internal Request as invalid.

- If the provided Opportunity is Closed Lost, you can let the requester know the Opportunity must be Closed Won and then close the Internal Request as invalid.

How does Cloud Licensing impact customers who have purchased multi-year GitLab subscriptions?

- If an existing or new customer purchases a multi-year subscription and has Cloud Licensing enabled then the Cloud Activation Code they receive will be valid for the entirety of the subscription period.

- If an existing customer has already activated a subscription with an Activation Code then the customer will not have to do anything, the license for the subscription will automatically update during the sync process.

- If an existing customer has not yet enabled/activated Cloud Licensing (TurnOnCloudLicensing set to Null) they can activate their instance with an Activation Code at any time

- If an existing customer who previously was undecided (i.e. Turn On Cloud Licensing is set to Null) or opted out of Cloud Licensing (i.e. Turn On Cloud Licensing is set to No or Offline) wants to switch to Cloud Licensing, they will need to contact support. Support can generate and send the activation code in CustomerDot Admin as detailed in post sales exemption support.

- Upon activation, the customer will receive a Cloud Activation Code which will be valid for the entirety subscription term

Additional Resources

For more information on Cloud Licensing, please refer to the following resources:

1. Cloud Licensing Internal Handbook Page
1. Offline Cloud Licensing Internal Handbook Page
1. Cloud Licensing Field Team FAQ

Have a question that hasn't been answered in the documentation? Add it to our Cloud Licensing AMA doc (internal only)!

SECTION: support/license-and-renewals/workflows/self-managed/creating-wider-community-license.md

Overview

GitLab allows community contributors to contribute to GitLab Enterprise Edition. Wider community members should follow the process described here, this page is targeted at GitLab team members. After trialling for 30 days, an EE license for 90 days can be generated if requested by a Contributor Success team member using the internal request form. Use the GitLab Support Internal Requests for Global customers request option, and Wider Community License for the internal request type. Subsequent requests can be processed as necessary.

Actioning

An internal request will be received similar to this example, which should contain the required information to generate a license. Follow the procedure to create a new license, ensuring the following:

- Plan: Ultimate, unless specified otherwise.
- Trial: Yes (is ticked)
- Starts at: (The current date)
- Expires at: (90 days or 1 year from the current date)

Responding

Once you've generated the license, send a public response on the internal request including the requestor, who will be CC'd, confirming that the license has been generated and sent directly to the contact on the email address provided.

SECTION: support/license-and-renewals/workflows/self-managed/creating_licenses.md

Overview

You will frequently need to generate a new license key due to either user error or system error.

Self-managed licenses are managed in CustomersDot.

Access to the application is managed through Okta, in order to request access please open an access request and refer that your role entitles you to access with this link.

License types

An overview of the different licenses types can be seen in this handbook page (internal only).

The type of the License object associated to a subscription version can be identified by using the info below:

License type	Turn On Cloud Licensing
Cloud License	'Yes'
Offline License	'Offline'
Legacy License	'No' or null

Duplicate a license

To re-issue a license:

1. Log in to the CustomerDot.

1. Identify the license by searching by customer email or company name.

1. Click Duplicate license button on the right (looks like the copy symbol).

Note: Only Legacy Licenses can be duplicated as of today.

1. Select the correct license type (see License Type Overview for more info).

1. Modify the values you need. Be careful not to change the Customer field.

Note: Changing the Customer causes the inputs for name, email and company to be overridden on creation with the selected customer's information. This is a bug that will be addressed in <https://gitlab.com/gitlab-org/customers-gitlab-com/-/issues/3566>.

1. In the Notes field, describe why you're issuing this license and add a link to the Zendesk ticket or GitLab issue for the request if applicable.

- If working on a GitLab issue, add the label Manually Generate Trial License (for trials) or Manually Generate License.

1. Click Save.

The license should be emailed out immediately. Please note that a license should always be sent to the end user.

Do not email the license to a GitLab team member or a reseller.

Note: If you want to send the license to a different email address use this handbook page instead.

When an existing license isn't available to duplicate, use the create new license section.

Create a new license

To create a new license, click Add new License then follow the process listed below for creating a Legacy License or Offline Cloud License.

Note:

- Depending on the selected license type, different fields are required. Please see the required fields section

for each license type for more information.

- When filling in the Zuora subscription ID (depending on the type's requirement), the subscription's existence is checked against Zuora and will fail if it's not found.

- When the Zuora subscription ID is filled, the subscription name will be automatically set on license creation.

Create a Legacy License

1. Select Legacy License in the License type drop down.
1. Fill in the Zuora subscription ID.
Note: These fields are not required for Legacy Licenses but should be filled out by Support.
1. Fill in other fields.
1. In the Notes field, describe why you're issuing this license and add a link to the Zendesk ticket or GitLab issue for the request if applicable.
 - If working on a GitLab issue, add the label Manually Generate Trial License (for trials) or Manually Generate License.
1. Click Save.

Required fields

Currently the following fields are required for a Legacy License:

- License type
- Name
- Company
- Email
- Users count
- Plan code
- Starts at
- Expires at

Required to be filled by Support:

- Customer
- Zuora subscription ID
- Notes

Create an Offline Cloud License

When creating an Offline License, the following has to be present beforehand:

- The filled in Zuora subscription has to be cloud license compatible (the Turn on seat reconciliation field is present).
- The filled in Zuora subscription has to be an Offline Cloud License or a disabled Cloud License (Turn On Cloud Licensing set to Offline or No).
- An activation code has to exist for the subscription (usually created via a callback to CustomersDot when the Zuora subscription is created).

1. Select Offline Cloud in the License type drop down.
1. Select the customer in the Customer field.
Note: This field is required for Offline Cloud Licenses.
1. Fill in the Zuora subscription ID.
Note: These fields are required for Offline Cloud Licenses.

1. Fill in other fields.

1. In the Notes field, describe why you're issuing this license and add a link to the Zendesk ticket or GitLab issue for the request if applicable.

- If working on a GitLab issue, add the label Manually Generate Trial License (for trials) or Manually Generate License.

1. Click Save.

Required fields

Currently the following fields are required for an Offline Cloud License:

- License type
- Customer
- Name
- Company
- Email
- Zuora subscription ID
- Users count
- Plan code
- Starts at
- Expires at

Required to be filled by Support:

- Notes

Create an Online Cloud License

With Strict Cloud Licensing, Online Cloud Subscriptions only send out an activation code. A license will no longer be supplied. Hence, license creation for this type is not supported.

SECTION: support/license-and-renewals/workflows/self-managed/gcp_marketplace_license.md

GCP Marketplace License Overview

Customer can purchase GitLab subscription from GCP Marketplace. This purchase allows the customer to pay for the subscription monthly.

Even though, the customer is paying for the subscription monthly, they signed an annual or multi-year contract with monthly payments.

So they cannot reduce the number of User mid-term. They can only reduce at renewal time - which would mean churn.

The customer purchased via GCP counted as reseller customer so they would not have access to the customer portal and there is no way to add more seats from GCP either, see GitLab issue 4930. To make change to the subscription, the customer would need to reach out to Sales team.

- Example ticket request to reduce seats
- Example ticket request to add seats

SECTION: support/license-and-renewals/workflows/self-managed/generating_hackerone_reporter_licenses.md

Overview

As part of GitLab's HackerOne program policy, a reporter who has submitted three or more valid findings to the program is eligible to receive a one-year self-hosted Ultimate license, supporting up to five users. The request will originate from someone following the Awarding Ultimate Licenses runbook.

HackerOne at a corporate level has a triage team that GitLab employs for bug bounty triaging and validation. To aid them with performing this work we issue them 2 Ultimate licenses on a yearly basis. These licenses are generated as part of the HackerOne Triage Team GitLab licenses runbook and are sent to analysts@managed.hackerone.com and analysts+1@managed.hackerone.com. Since the license does not change from year to year, the existing licenses can be duplicated for the purposes of the workflow. When duplicating either of the licenses, please ensure the dates are adjusted for the year to come and the note is replaced with a link to the ticket in which the license is being requested. As with other legacy licenses that are manually generated, each license needs to be requested in its own ticket for compliance and auditing purposes.

Actioning the Request

The internal request will come through similar to these examples and should include:

- The contact's name, which will either be a full name OR their HackerOne 'handle'
- The contact's email, which should end with a @wearehackerone.com suffix.

With this information, follow the procedure to create a new license, and ensure you use the following:

- Name: (The contact's full name OR HackerOne 'handle')
- Company: H1 Reporter Award
- Email: (The contact's email)
- Users count: 5
- Plan: Ultimate
- Trial: Yes (tick the checkbox)
- Starts at: (The current date)
- Expires at: (One year from the current date)
- Notes: (Include a link to the internal request ticket)

See this generated license for an example of the completed license.

Responding to the Request

Once you've generated the license, send a public response on the internal request including the requestor, who will be CC'd, confirming that the license has been generated and sent directly to the contact on the email address provided.

SECTION: support/license-and-renewals/workflows/self-managed/generating_nfr_licenses.md

Overview

As part of GitLab's NFR program policy, Channel Partner team will open internal request for NFR license using the internal request form.

As of Feb. 19, 2025, partner NFR licenses will be provisioned as standard cloud licenses instead of as trial subscriptions. Ecosystem Operations manage this process and no longer requires assistance from the support team to provision. Reach out to global-ecosystem-programs-ops via slack with any questions.

SECTION: support/license-and-renewals/workflows/self-managed/handling_flagged_provisioning_failures.md

The Provision tracking system provides a means of monitoring provision failures via a failure alerting system in the Slack channel provisionfailures so that the Fulfillment Product Team and the L&R Support Team can be made aware quickly when a failure occurs and act upon it.

Workflow for handling Provisioning failures

1. The Fulfillment Team will monitor the Slack channel for provisioning failures. The following link provides information on the Provision Failure Monitoring process from the Provisioning team perspective.

1. If the Fulfillment Team is unable to resolve a failed provision then they will request help from L&R Support via the internal request form. Use the GitLab Support Internal Requests for Global customers request option, and Report a provision failure for the internal request type. The criteria for submission are:

- If Fulfillment requires L&R Support intervention to fix something for the customer, OR
- The root cause of the issue is already fixed or being fixed, so it's not going to be an ongoing failure case

1. In such situations the Support Engineer should open a ticket on behalf of the customer to inform them of the issue and offer them a solution or workaround if one is available. The Support Engineer can communicate if required with the Provisioning submitter via the raised Zendesk ticket.

1. If required, the Support Engineer can also submit a new bug issue, update (requesting higher prioritization or feedback) an existing issue or create a new feature request as outlined in the GitLab handbook section Managing Product Issues

SECTION: support/license-and-renewals/workflows/self-managed/handling_multi-

years_subscription.md

From time to time, you will run into cases where customer has a multi-year subscription. This is only possible if a subscription is purchased through Sales, since there's no option to make a multi-year subscription from CustomersDot.

Problems with multi-year subscriptions

1. Multi-year subscriptions on a legacy license are automatically generated for the first year only. Multi-year subscriptions on cloud licensing and offline cloud licensing are generated for the full subscription term. Read more at [Licensing FAQ](#).

Workflows

Confirm customer subscription

To check whether the customer has a multi-year subscription:

1. Navigate to Salesforce and locate the relevant opportunity which generally has a Close date in the recent past
1. Click on the opportunity · search for quotes · click on the most recent quote
1. Search for Renewal Term, which shows the subscription term for the quote in months

Handling the request

After confirmation and before generating a new license with a proper Start date and End date for the customer, please ask the customer for their system information using the ZenDesk Subscriptions::Active Users macro.

1. If there are not any Users over license, proceed to generate the new license.
 - Please note: licenses for multi-year subscriptions are issued in 12 month blocks
 - Amend the dates to match the relevant time interval - for example, the second annual license for a multi year subscription would have the original start and expiry dates, with an additional year added to both.
 - For example, Start date: 2021-05-01 and End date: 2022-05-01 would become 2022-05-01 and 2023-05-01, respectively.
 - Do not tick the trial checkbox, as this is not a trial!
 - Use the previous year license Users count as the new license Previous Users Count
1. If there are Users over license, confirm that they are accounted for in the quote. Otherwise, follow the Working with sales workflow. We can generate a new license once the Users over license has been paid for.

SECTION: support/license-and-renewals/workflows/self-managed/how-to-handle-gitlab-dedicated-licensing.md

Workflow for handling GitLab Dedicated Licensing

GitLab Dedicated license provisioning follows the same process as Self Managed subscriptions including multi-year subscriptions.

An activation code is automatically created and sent to customer upon deal close.

Refer to this internal handbook for more information.

Requests to forward or send GitLab Dedicated activation codes

Any requests to forward or send GitLab Dedicated licenses or activation codes should follow the same policies and workflow when sending licenses to a different email address for Self Managed subscriptions.

SECTION: support/license-and-renewals/workflows/self-managed/license_delivery.md

Overview

The support team is responsible for generating licenses when the automated systems are unable to do so or an error is made. This is done via CustomerDot and the Licenses section.

Generating a license may originate from an internal request or may be a required action to help move a support ticket forward.

In both cases, CustomersDot will automatically email the license to the customer directly after creation. Unless the circumstances require, we should not send the licenses to any GitLab employee. Exceptions may include air-gapped installs or other situations where the customer cannot receive the license via the email.

Note that a license should always be generated with the sold-to contact's name and email address. A license should not be generated with reseller or GitLab team member details if the license is for a customer.

Where does a license get sent to?

There is often confusion about the email address that the license will be delivered to and how to ensure the correct email address receives the license. With the work around aligning CustomersDot BillingAccounts and Users to Zuora objects, this became much simpler:

The license will be sent to the sold-to contact in Zuora with all sales (code reference). The email for the sold-to contact can be changed via a Zuora Sold to contact change.

Caveats

1. Resending an activation code via the CustomersDot Admin uses the email address of the sold-to contact in Zuora.

1. Resending a legacy license or offline cloud license via the CustomersDot Admin uses the email set in the license regardless of the sold-to contact in Zuora.

1. Forwarding an activation code, a legacy license or offline cloud license via the CustomersDot Admin uses the email entered in the Destination email address field.

Check whether the license has been delivered

Email delivery is logged for 30 days in MailGun. If the license was generated more than 30 days ago, we will not know whether the license was delivered or not. To see email license delivery log:

1. Follow the instruction in checking mailgun

1. Use customers.gitlab.com as Domain in Step 4 in the check mailgun instruction

1. Update the date range to cover the license generation date

SECTION: support/license-and-renewals/workflows/self-managed/license_for_weekend_emergencies.md

Overview

There are times when a customer submits an emergency support request for a new license to replace their expired one immediately. But on-call Support Engineers do not usually have access to CustomersDot, and cannot manually generate licenses.

This workflow describes steps that can be followed by anyone in Support, regardless of whether they have CustomersDot access, to generate a short-term temporary Ultimate Trial license to de-escalate a weekend emergency.

NOTE: This workflow is applicable even if the customer has previously received a Sales-generated temporary extension.

This workflow does not cover SaaS Subscription Emergencies, see Customer Emergencies Workflow - SaaS License Emergencies.

Scope

License requests for a Self-managed customer with a paid plan, where the license's "grace period" (14 days after expiration) ended within the last 3 days from the current emergency's date.

Out of Scope

1. Organizations without a paid plan.

1. Prospects.

Workflow

Step 1: Confirm customer subscription

1. Request from the customer the following screenshots:

- Admin Area -> Overview -> Dashboard
 - The license page from Admin Area -> License (in newer versions, may say subscription)
1. Add the screenshots to the ticket.

Step 2: Generate the trial license

1. Go to the Mechanizer ZenDesk App, from the ticket in the right side bar.
1. Enter your username and the user count for the license based on information from the screenshot.
1. Enter the email address of the person who opened the ticket.
1. The tool generates and emails to the customer a 10-day Trial GitLab License for the given user count.
1. Post a comment in the supportlicensing-subscription Slack channel, with the ticket url, to let the team know that an emergency license was generated.

Considerations

1. All licenses generated with the tool will have a 10 day limit. This cannot be changed through the tool.
1. The license will have generic information. This is intended to facilitate its creation.
1. If the customer mentions that they haven't received the email with the license, go to our internal request issue tracker and you will see a closed issue with the title: `[Success] - Created a license under [EMAIL] for [X] users`. That issue will have a copy of the license. If there aren't any closed issues, you can look for an open issue which will have information about the failure. If you find it and no license was generated, you can use the tool again to generate another license.

SECTION: support/license-and-renewals/workflows/self-managed/opt-out_service-usage-data.md

Overview

The Support team doesn't have the ability to opt the customer out of Services Usage Data and the customer cannot opt-out of the feature without Sales team approval as mentioned in the handbook. To handle this request, you can

1. Find the Account owner on SFDC
1. Use the Zendesk Macro Support::Self-Managed::Super Sonics Opt Out and fill in the account information
1. Submit as Solved

SECTION: support/license-and-renewals/workflows/self-managed/sending_license_to_different_email.md

Overview

A temporary license can be sent to a different email address.

A paid license is always sent to the Sold to contact used on the order, and this contact is mentioned in the license file.

We can send the license to another contact in either of the following cases:

- The contact has a Customers Dot account which is linked to the subscription.
- The contact is the Sold to contact for the Zuora account.

To send the license to a different email address, follow the process for Associating purchases with additional accounts. \

We cannot bypass doing this based on internal requests. It must be done via a customer submitted ticket. \

Once the new Customers Dot account is created or changed, the license can be re-sent (or obtained by the new account owner).

The Support team is not allowed to make changes to the contact information in the license itself.

To forward or resend a license or activation code:

- Find the license or activation code by searching using customer email or company name.
- To resend the license or activation code to the same user, click the Resend to customer via email button on the right. (Looks like an envelope icon).
- To forward the license or activation code to a different address using the Forward license email function:
 1. Navigate to the Forward license email tab if it's a license file or to the Forward Cloud activation email tab if it's an activation code.
 1. Enter the Destination email address. NOTE It is currently not possible to copy (cc) or send to multiple contacts at once.
 1. Click the Forward button.
 1. Confirm whether the license was delivered

SECTION: support/license-and-renewals/workflows/self-managed/trials.md

Overview

Self-managed trials and grace periods cannot be extended - a trial license must be issued and applied to the instance in order to "extend" a license.

Requests for grace period extensions, temporary keys, temporary extensions, temporary licenses, and trial extensions all require generating a trial License.

Sales will often request through an Internal Request / Zendesk Ticket that we extend the duration of Self-Managed trials on behalf of their prospects. These tickets will always be raised from the GitLab Support End User gitlabsupport@example.com, with the submitter cc'd on the ticket. The following workflow should be followed to service them.

If any fields when opening the ticket were filled out incorrectly, send a public reply in the ticket asking the submitter to supply the missing information.

> NOTE: Non-trial licenses are required to match an existing subscription and these licenses generally have a span of 1 year. There is an ongoing discussion on how to support complex subscription models.

Support tries as much as possible to refrain from issuing non-trial licenses. We are allowed to issue trial licenses because they:

- Have no grace period
- Do not affect historical data tracking
- Are not tied to any subscriptions for revenue purposes

How to extend an expired or soon to expire license

Note: Unlike SaaS, a Self-managed extension does not require a pre-existing Trial license. Instead, start with the current almost-expired license, or most-recent expired license.

1. Open the expired or soon to expire license.

1. Confirm that:

1. The license is a non-trial license and:

- If the license expires within the next 15 days, redirect the salesperson to use the SFDC feature by using the Deviation from SM License Extension Workflow macro and close the ticket.

- If it expires more than 15 days later, the salesperson should wait until the expiry is within 15 days and then use the SFDC feature. Redirect the salesperson to use the SFDC feature by using the Deviation from SM License Extension Workflow macro and close the ticket.

- If the license expiry has passed, then proceed with step 2 (2).

1. The customer does not have a valid temporary extension:

1. Go to <https://customers.gitlab.com/admin/temporaryextensionhistory>

1. Search using the Subscription Name that was in the license.

1. If a temp extension exists and its expiry date is not within 5 days, tell the salesperson to open the request when the license will expire within 5 days and close the ticket.

1. If a temp extension exists and its expiry date is in 5 days or less, proceed to step 2 (3).

1. If a temp extension does not exist, proceed to step 2 (3).

1. The ticket type is Extend an (almost) expired subscription.

- If the salesperson selects a different form (e.g. the IR Other), then redirect them to submit a new ticket by using the Deviation from SM License Extension Workflow macro and close the ticket.

- The I acknowledge that approval for this extension has been granted.. checkbox has been checked and the requestor has provided the required proof that a Manager or Director has approved the extension. If the submitter has not provided the necessary proof then use the macro Deviation from SM License Extension Workflow and subsequently close the ticket.

- If the ticket type is correct and the license expiry has passed, then also proceed with step 3.

1. Select Duplicate License.

1. Set the License type to Legacy License.
1. Delete the contents of Zuora subscription ID field if present.
1. Set the Users count number to what is requested.
1. Set the Previous users count number to the previous license if there are trueups OR if the requested User Count is less than the Previous User Count. Otherwise, delete the contents.
1. Set the Trueup count number if present.
1. Set the Plan code to what is requested.
1. Ensure the Trial checkbox is checked.
1. Set Starts at to today's date.
1. Set Expires at to the requested date (Expires at 0:00 on this date).
1. Set Notes to the ticket or issue URL.
1. Click Save. The license will be automatically sent to the email specified in the Email field.

If you need to send a trial license to another contact, use the Forward license email tab after saving the new license.

How to create a new trial license

Users should initiate a request on their own by clicking on the following link:
<<https://about.gitlab.com/free-trial/?hosted=self-managed>>

Emergency Weekend Licenses

If you're on call and you need a license generated, but don't have access to the CustomersDot interface, follow the Weekend Emergencies - License Request workflow.

SFDC generated trial license extensions

Account Executives (AEs) can now use Salesforce.com (SFDC) to issue a self-managed 21-day license extension to a customer when the renewal opportunity is taking longer than expected to close. When an AE uses this functionality, the license code is automatically generated and sent to the customer without any L&R Support involvement. The Temporary renewal extensions handbook entry documents this approach. Note that there are guardrails in place to prevent abuse of this ability, and that as a result only one license extension can be generated per renewal event. Therefore on occasions L&R Support will still be required to generate further license extensions.

Sales AE requests for a manual temporary extension

An AE is expected to use the SFDC process first when a customer requires a self-managed license extension. However, if an extension has elapsed then an AE can use the Internal Request Form to request a license extension directly from L&R support. Use the GitLab Support Internal Requests for Global customers request option, and Extend an (almost) expired subscription for the internal request type. There are some caveats to this process which are listed below:

- If the AE selects a different form (e.g. the IR other), then the L&R Support engineer should redirect the AE to submit a new ticket by using the Deviation from SM License Extension Workflow macro and close the ticket.
- If the AE has not first utilized the SFDC process, then the L&R Support engineer should

direct them there by using the Deviation from SM License Extension Workflow macro and close the ticket.

The Deviation from SM License Extension Workflow macro is to be used to record an instance where there has been a deviation from the process for creating a self-managed license extension for a customer. When a customer needs a short-term extension for a self-managed subscription, the assigned account executive should initially utilize the SFDC process to automatically produce a temporary self-managed license, valid for 21 days. This enables the client to maintain access to GitLab while their subscription is being renewed. This macro should be used in situations such as the following:

- Sales/CSM has selected the incorrect form
- Sales/CSM approached Support out of Zendesk, via issues or Slack
- Sales/CSM provided incorrect or incomplete information
- Sales/CSM raised an IR instead of first using SFDC to generate a license extension

Customer requests for a license extension

When a customer requests a license extension, follow the steps below based on the day of the week and whether the customer is classified as Enterprise or SMB.

1. Locate the Account Owner:

- In the Zendesk (ZD) ticket, look for the internal note labeled Organization Info to identify the Account Owner:
 - For Enterprise/Commercial customers, a specific individual will be listed.
 - For SMB customers, you will see EMEA/AMER/APAC SMB Sales instead of a named person.

2. Determine next steps Based on the Day and Customer Type:

Day Enterprise/Commercial SMB
-- ----- -----
Weekday Redirect to Sales Redirect to Sales
Weekend/Holiday Redirect to Sales Issue temp extension and redirect to Sales

3. Steps for Redirecting to Sales:

Enterprise/Commercial Customers:

- Apply the Customer Request for SM License Extension macro, updating the template to include the AE's email address before sending it to the customer.
- Notify the Account Executive (AE) through Chatter to ensure they are aware of the request.

SMB Customers:

- Follow the process outlined in the Working with the Global Digital SMB Account Team handbook page.
- Provide the Salesforce (SFDC) ticket ID to the customer.

4. For SMB Customers on Weekends or Holidays, please issue a temporary license extension before redirecting to Sales.

Licensing pathways for handling customer renewals and new sales that have become delayed

In certain scenarios where customer renewals or new customer sales are experiencing delays, the L&R Support process workflows provide flexibility to address these challenges. The following table outlines the options available to issue temporary trial licenses based on specific use cases:

Use Case	Pathway
-----	-----
Customer renewal is taking longer than expected	The sales AE (Account Executive) generates a one-off 21 day temporary renewal extension via SFDC
Customer renewal exceeds the additional 21 days	The sales AE can open a new Internal Request (IR) ticket with L&R support and request a trial license extension for up to 1 month
Customer renewal exceeds the additional 21 days + 1 month	The sales AE can open a new Internal Request (IR) ticket with L&R support who request approval via the ticket from the senior director of revenue @andrewmurray
New customer potential sale	The sales AE can request up to 1 month trial extension via an IR with L&R support.
New customer sale taking longer than 1 month	The sales AE generates a \$0 dollar opportunity in SFDC, then opens a new IR ticket with L&R support who request approval via the ticket from the senior director of revenue @andrewmurray

SECTION: support/license-and-renewals/workflows/self-managed/troubleshoot_cloud_licensing.md

Scenario

Customer was opted into cloud licensing because of a bug and their instance activated cloud licensing, an entity change happened and the new subscription was not on cloud licensing

Troubleshooting workflow

1. Determine whether the customer should be on cloud licensing

1. If no, proceed with:

1. Removing the current cloud license by asking the customer to run these commands via rails console: `l = License.find 123` and then `l.destroy`. You need to provide the license number for the first command. You can find this number in the URL of the cloud license in the customersdot license menu (example: `<https://customers.gitlab.com/admin/license/123123123>`)

1. Once the license has been removed, the customer should have the ability to upload a standard license file on their instance.

1. If yes, proceed with:

1. Switching the new subscription to cloud license features

1. Removing the current cloud license by asking the customer to run these commands via rails console: `l = License.find 123` and then `l.destroy`. You need to provide the license number for the first command. You can find this number in the URL of the cloud license in the customersdot license menu (example: `<https://customers.gitlab.com/admin/license/123123123>`)

1. Once the license has been removed, the customer should be able to input the new activation code that was generated and emailed to them when you switched the subscription to cloud features.

1. See example ticket where customer did want to be on cloud licensing

How to switch a subscription to cloud license features

1. Go to the customer's Zuora Subscriptions tab in customersdot (URL example: <https://customers.gitlab.com/admin/customer/123123/zuorasubscriptions>)
1. Tick the relevant feature boxes. DO NOT tick the Cloud Licensing checkbox. Click the Update button.
1. Click on the cloud icon next to the update button to generate an activation code for the subscription and email it to the customer.
1. Check that a code has been generated by viewing the Cloud Activations tab. Ensure that the code was generated for the correct subscription.
1. Impersonate the customer's portal account to ensure that the activation code is available for the subscription.
1. Once the customer has used the activation code on their instance, a cloud license should have been generated (check the licenses section) and the Cloud Licensing checkbox from step 2 should now be checked.

Troubleshooting Network Connectivity

Cloud licensing requires a connection to customers.gitlab.com over port 443 (HTTPS), and this connection must remain available throughout the duration of using a cloud license. The GitLab server will typically check in once per day, as well as once during activation, and any time a manual sync is performed. While general networking is typically outside the scope of what we can support, there are a number of things we can easily test for to help users diagnose any potential network or HTTPS issues blocking the connections.

Check DNS

The connection will rely on the hostname customers.gitlab.com, meaning a DNS lookup will be performed. The IP addresses of customers.gitlab.com may change periodically, so it's important to verify what they are right now:

For example, using dig) or host commands:

```
sh
dig customers.gitlab.com
```

```
host customers.gitlab.com
```

Keep in mind, IPv6 may occasionally be a factor in network troubleshooting. If a server is configured to prefer IPv6 over v4, then be sure that their server is able to resolve IPv6 (AAAA) records:

```
sh
dig AAAA customers.gitlab.com
host -t AAAA customers.gitlab.com
```

Basic IP and TCP connectivity

In most cases, a ping will suffice to verify that a connection is available over an IP address, be it IPv4 or v6. After obtaining the current IP addresses:

```
sh
ping 104.18.21.224
ping -6 2606:4700::6812:14e0 ping6 may be available too, but is an alias for ping -6
```

Pinging the hostname (customers.gitlab.com) is fine too, but directly at an IP ensures both IPs are available. A simple one liner can be used to ping both IPs:

```
sh
dig +short customers.gitlab.com | xargs -l% ping -c1 -W5 %
```

Note that ping is not technically a TCP or UDP connection, but rather an ICMP, which may occasionally be blocked for various networking reasons. A slightly more advance technique can be used to test TCP connections, using either nc (preferred) or telnet:

```
sh
$ nc -zv customers.gitlab.com 443
```

Connection to customers.gitlab.com 443 port [tcp/https] succeeded!

```
sh
$ echo "." | telnet customers.gitlab.com 443
```

```
Trying 104.18.20.224...
Connected to customers.gitlab.com.
Escape character is '^]'.
Connection closed by foreign host.
```

Verifying HTTP/S connections

The simplest method to check this is by using curl. Specifically, sending a HEAD request to https://customers.gitlab.com, which will both verify TCP connectivity and ensure a secure TLS connection is available:

```
sh
curl -I https://customers.gitlab.com/
```

A successful reply will generally look like a 302 (Found), meaning the connection was succesful, and customers.gitlab.com replied with a redirection. More concisely:

```
sh
$ curl -IL -q https://customers.gitlab.com/ | grep -i location
```

location: <https://customers.gitlab.com/customers/signin>

Implies success.

Troubleshooting SSL/TLS

In most circumstances, if system curl is working, then there won't be a need to check further TLS settings. However, because GitLab does offer several options for configuring various TLS and SSL certificate settings, including custom certificate authorities, it may be necessary to perform some more advanced SSL/TLS troubleshooting.

For example, if the customer server is making use of custom certificate authorities (CA), such as when SSL packet inspection is employed, they will need to add that root CA certificate to `/etc/gitlab/trusted-certs` on the server, then reconfigure GitLab.

- Refer to our documentation on how SSL works in GitLab and troubleshooting SSL in GitLab for more in-depth information. In particular, relying on the packaged version of openssl to check and verify SSL connectivity:

```
sh
```

```
echo | /opt/gitlab/embedded/bin/openssl sclient -connect customers.gitlab.com:443
```

It may be useful to run a SSL Server Test and then provide the page to customers. It's beyond the scope of our support to assist with most of the output on this page, but providing it may prove useful to customer network admins for additional troubleshooting on their end.

Obtain DevTools > Network HAR file during activation

A user's browser's DevTools can be particularly useful in diagnosing cloud license connectivity failures, especially since the GitLab internal API (graphql) response can be viewed.

1. Open the DevTools (usually `ctrl+shift+i`) and navigate to the Network tab

1. (re)Load the page at `/admin/subscription`

1. Attempt to activate the cloud license code

1. In DevTools > Network, locate any graphql resources and view the Response pane. There may be errors recorded here, feel free to share.

- Note that multiple graphql resources may be present, and not all will be related to the cloud licensing activation process.

Since there will be a lot of information presented in the DevTools, feel free to suggest that the customer generate a network HAR file and attach it to the ticket for closer inspection by us.

Caution: Advise the user to sign out of the GitLab session they recorded to invalidate their session credentials.

See [sec.Okta.com/harfiles](https://sec.okta.com/harfiles) for context.

Simulating a cloud license SeatLink attempt from within the Rails application

System tools and binaries such as ping and curl are helpful, but keep in mind that GitLab is a highly complex application that bundles many of its own tools and binaries. GitLab the application may handle outbound connections slightly differently than system-supplied binaries will. The best way to verify this is to check that the connection is possible directly from rails console. Login to the server and enter rails console with `sudo gitlab-rails console` and run:

```
ruby
URIPATH = '/api/v1/seatlinks'
baseuri = EE::SUBSCRIPTIONSURL For GitLab version lower than 16.0.0
baseuri = ::Gitlab::Routing.urlhelpers.subscriptionportalurl For GitLab version 16.0.0 and
above

headresp = Gitlab::HTTP.head(baseuri)

pp headresp
```

That command will return output very similar to what a typical `curl -I` would return, but it relies on internal ruby classes to route the request in the same way GitLab would, therefore exposing any potential issues that are happening specifically within the GitLab application.

Simulating a cloud license SeatLink attempt with curl

If you need to check if the SeatLink query will work from a different network segment, where GitLab is not installed, you may use Rails console to generate an equivalent curl command. To do that, enter the Rails console with `sudo gitlab-rails console` and enter the code snippet below:

```
ruby
headers = Gitlab::SubscriptionPortal::Client.send(:jsonheaders)
baseurl = Gitlab::SubscriptionPortal::Client.send(:baseurl)
params = Gitlab::SeatLinkData.new.tojson
path = "api/v1/seatlinks"
curlcmd = ['curl --trace - --trace-time -X POST']
curlcmd << "\"{File.join(baseurl, path)}\""
headers.each do |key, value|
  curlcmd << "-H \"{key}: {value}\""
end
curlcmd << "-d '{params}'"
puts curlcmd.join(" \\\n ")
```

The result should look similar to the one below:

```
shell
curl --trace - --trace-time -X POST \
  "https://customers.gitlab.com/api/v1/seatlinks" \
```

```
-H "User-Agent: GitLab/17.6.1-ee" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{"gitlabversion":"17.6.1-ee","timestamp":"2025-01-21T12:00:11Z","licensekey":"xxx...\n","maxhistoricalusercount":2,"billableuserscount":2,"hostname":"gitlab.example.com","instanceid":"aaaaaaaa-0000-0000-aaaa-aaaaaaaaaaaa","addonmetrics":{"addontype":"duoenterprise","purchasedseats":10,"assignedseats":3}}'
```

Notice the `--trace` - `--trace-time` parameters - this will cause curl to produce large amounts of debug output. You can also add `--proxy [protocol://]host[:port]` if needed.

Custom proxy settings

At present, cloud licensing does not officially support network proxies, deep packet inspection, etc. But if the customer is aware of custom network proxy configurations on their end, they may be able to configure GitLab to ignore them via the `noproxy` environment variable. More information is available in our documentation on [Setting custom environment variables](#)

Activating a license with the GraphQL API

In cases where the license activation functions are unavailable (for example, due to a 500 error on the billing page), the GraphQL API can be used to activate a cloud license directly using the `Mutation.gitlabSubscriptionActivate` endpoint. With this method, a subscription can be activated even if there is already an activated cloud license present on the instance. This method is useful to avoid the downtime caused by removing the existing license via the UI before activating another key:

1. Have the customer navigate to <https://<their-self-managed-gitlab-site.com>/-/graphql-explorer>
1. Run the following mutation by replacing `<activation code>` with the actual 24-character cloud activation code:

```
graphql
mutation {
  gitlabSubscriptionActivate(
    input: {
      activationCode: "<activation code>"
    }
  ) {
    errors
    license {
      id
      activatedAt
      startsAt
      usersInLicenseCount
      lastSync
    }
  }
}
```

The mutation will return the following if there are no errors:

```
graphql
{
  "data": {
    "gitlabSubscriptionActivate": {
      "errors": [],
      "license": {
        "id": "gid://gitlab/License/12345",
        "activatedAt": "2023-07-28",
        "startsAt": "2023-07-04",
        "usersInLicenseCount": 10,
        "lastSync": "2023-07-28T18:34:14Z"
      }
    }
  }
}
```

General Tips

1. You can resend an activation code via email by clicking on the envelope button on the customer's Cloud Activations tab.

SECTION: support/license-and-renewals/workflows/self-managed/troubleshoot_license_purchase_issues.md

Customers may report encountering the following error message when purchasing additional seats for their licensed GitLab Self-Managed instance:

> The Charge is not allowed to be updated on MM/DD/YYYY

This is caused by a customer renewing their license more than once so the newest license starts at a future date more than a year into the future and then refunding the future dated license. The future dating in the system causes problems with the present such as being able to add additional seats.

Example

- Customer renews their license for 2020-03-01 to 2021-03-01 then renews again for 2021-03-01 to 2022-03-01
- The newest license for 2021 to 2022 becomes the only available license for download
- Working with support and accounts receivable the customer requests a refund on the newest license
- Due to the nature of the Zuora billing system this creates an effective future date for amending the existing license which cannot be changed
- The customer now sees the error The Charge is not allowed to be updated on 03/19/2021 when

trying to add seats

- To fix this the accounts receivable team need to go into the subscription and completely rebuild the subscription in Zuora by creating a 0 invoice for internal purposes that the customer never sees

SECTION: support/license-and-renewals/workflows/self-managed/troubleshoot_license_upload_issues.md

Customers may report encountering the following error message when uploading a license to their GitLab Self-Managed instance:

> During the year before this license started, this GitLab installation had 286
> active users, exceeding this license's limit of 250 by 36 users. Please
> upload a license for at least 286 users or contact sales at [redacted]@gitlab.com.

This error is shown if any of the following 3 values are incorrect:

| Key Field | Definition & source of truth |

|-----|-----|

| Users count | GitLab 13.6 and later We can identify the minimum amount via the billable.users command: `sudo gitlab-rails runner 'p User.billable.count'`

GitLab 13.5 and before We identify the minimum amount via the active.users command: `sudo gitlab-rails runner 'p User.active.count'` |

| Previous users count | We can try to assume based on internal numbers such as CustomerDot or usage. But this cannot be certain since multiple licenses may be generated for one period. Instead, the best identifier is the system itself. For this we request a screenshot of the user statistics panel from Admin Area -> Overview -> Dashboard. The Previous users count = Users in License in the widget on the top left. |

| True-up count | The best identifier for this value is to take Max Users - Previous users count. We recommend using the historical.max command: `sudo gitlab-rails runner 'p ::HistoricalData.maxhistoricalusercount'` for the Max count.

Important: As of GitLab 13.9 the maxhistoricalusercount method requires two arguments, a range of dates specified by from: and to:: `sudo gitlab-rails runner 'p ::HistoricalData.maxhistoricalusercount(from: "2020-01-01", to: "2021-01-01")'`. |

Note: For Ultimate licenses, guest users are not counted in the licensed user count. Therefore, in GitLab 13.5 and before, the User.active.count will be a greater number than the Active Users count displayed in the Admin UI dashboard. In GitLab 13.6+ User.billable.count will accurately exclude guests from the count. This can be confusing in a situation where the customer is downgrading from Ultimate or if they have an Ultimate trial, because they need a license for User.active.count instead of what is displayed for Active Users.

You can use the Zendesk Subscriptions::Active Users macro to ask the customer for this information.

Examples

EXAMPLE 1

- Subscription for 10 users
- During the subscription term incurs a max of 15 users
- Before renewal, blocks 3 users
- At renewal, the license must be generated as: Users count: 12 (or more), Previous users count: 10, Trueup count: 5

EXAMPLE 2

- Subscription for 20 users starts in December with only 12 active users in the system
- In March, 5 users are blocked leaving 7 active users (12 max users)
- In April, 5 new users are added making the active user count 12 (12 max users)
- In July, 10 more new users are added making the active user count 22 (22 max users)
- In August, 5 users are blocked making the active user count 17 (22 max users)
- At renewal, the license must be generated as: Users count: 17 (or more), Previous users count: 20, Trueup count: 2

EXAMPLE 3

- Ultimate subscription for 10 users
- During the subscription term, has 12 active users, including 2 guest users
- At renewal, customer downgrades to Premium license
- License must be generated as: Users count: 12 (or more), Previous users count: 10, Trueup count: 0

Workflow for setting the correct Zendesk Form Fields to track license activation failures

As detailed in the issue Create a process which provides the means of identifying the number of customers who are not able to activate their instance due to overage enforcement the Zendesk L&R Form fields have been modified to allow support engineers to categorize tickets where a customer has been unable to activate their license using either a license file or an activation code.

In such scenarios the L&R Support Engineers should follow the workflow below to categorize the ticket appropriately.

1. Field: L&R - Category

- Set to Self-managed license related matters

2. Field: L&R - SM Subcategory

- Set to I am getting errors applying my license or activation code

3. Field: Transactions Issue Type

- If the customer used a license file, then set to License troubleshooting (SM only)
- If the customer used an activation code, then